

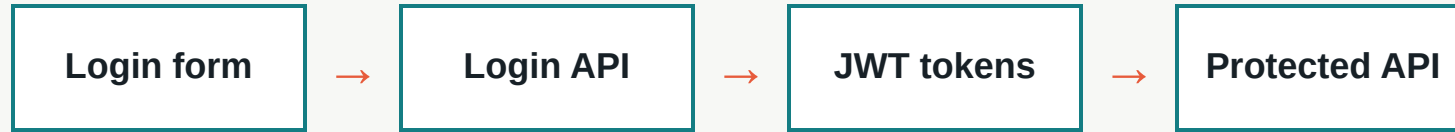
CLASS 3

JWT Login in Our Django CRM

Login, tokens, protected APIs, and the existing HTML page.

TODAY

What will we build?



A user logs in once, then proves identity using an access token.

BEFORE JWT

What did our login page do?

```
def login(req):  
    if req.method == "GET":  
        return render(req, 'login.html')
```

It only displayed the HTML page.

It did not check credentials, identify a user, or protect an API.

MEANING

What is JWT?

JWT = JSON Web Token

A JWT is a signed digital identity card returned after successful login.

Contains

User identity and expiry information.

Signed

Changes to it can be detected.

Not encrypted

Never store passwords inside it.

ARCHITECTURE

JWT login flow



The password is used for login. The access token is used for later API requests.

TWO TOKENS

Access token and refresh token

Access token

Short life. Sent to protected APIs.

Refresh token

Longer life. Creates a new access token.

Access expires



Send refresh



New access token

BEARER

How is the access token sent?

```
Authorization: Bearer <access_token>
```

Bearer means: “I am carrying this access token.”

Bearer is not an API. It is the authentication header format.

STEP 1

Install JWT support

```
pip install djangorestframework-simplejwt
```

Add to `requirements.txt`:

```
djangorestframework-simplejwt==5.5.1
```

SimpleJWT creates and verifies JWTs for Django REST Framework.

STEP 2

Create a Django login user

```
python manage.py createsuperuser
```

Django User

Authentication account with a hashed password.

Registration

Our CRM customer information model.

We use Django User for login because SimpleJWT is designed for it.

STEP 3

Tell DRF to recognize JWT

crm/crm/settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ),  
}
```

Now DRF can verify Bearer <access_token>.

STEP 4

LoginSerializer: receive login data

```
crm/registration/serializers.py
```

```
class LoginSerializer(serializers.Serializer):  
    email = serializers.EmailField()  
    password = serializers.CharField(write_only=True)
```

`write_only=True` means the password is accepted as input but not returned.

STEP 4 CONTINUED

Check credentials and create tokens

```
user = User.objects.filter(email=data['email']).first()

user = authenticate(
    username=user.username,
    password=data['password']
)

refresh = RefreshToken.for_user(user)

return {
    'refresh': str(refresh),
    'access': str(refresh.access_token),
}
```

STEP 5

LoginAPIView: control the request

```
crm/registration/views.py
```

```
class LoginAPIView(APIView):  
    permission_classes = [AllowAny]  
  
    def post(self, request):  
        serializer = LoginSerializer(data=request.data)  
        serializer.is_valid(raise_exception=True)  
        return Response(serializer.validated_data)
```

AllowAny: users can attempt login before having a token.

STEP 6

Add login and refresh URLs

```
crm/registration/urls.py
```

```
path('api/login/',  
     v.LoginAPIView.as_view(),  
     name='api_login'),
```

```
path('api/token/refresh/',  
     TokenRefreshView.as_view(),  
     name='token_refresh'),
```

One URL logs in; the other renews an access token.

STEP 7

Test the login API

```
POST /account/api/login/
```

```
{  
  "email": "teacher@example.com",  
  "password": "demo-password"  
}
```

Successful response:

```
{ "refresh": "...", "access": "..." }
```

STEP 8

Create a protected Profile API

```
class ProfileAPIView(APIView):  
    permission_classes = [IsAuthenticated]  
  
    def get(self, request):  
        return Response({  
            'id': request.user.id,  
            'username': request.user.username,  
            'email': request.user.email,  
        })
```

`request.user` is identified from the valid access token.

PROOF

Test protection both ways

Request	Result	Meaning
No token	401 Unauthorized	API blocks the request.
Valid Bearer token	200 OK	API identifies the user.
Expired/invalid token	401 Unauthorized	Token is rejected.

```
GET /account/api/profile/
```

STEP 9

Protect Registration CRUD

```
class RegistrationViewSet(viewsets.ModelViewSet):  
    queryset = Registration.objects.all()  
    serializer_class = RegistrationSerializer  
    permission_classes = [IsAuthenticated]
```

Now every Registration CRUD request needs a valid access token.

STEP 10

Connect the existing login page

```
const response = await fetch('/account/api/login/', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
});

localStorage.setItem('access_token', data.access);
localStorage.setItem('refresh_token', data.refresh);
```

Local storage is simple for this classroom demo, not the safest production design.

BROWSER FLOW

Use the token after login

```
fetch('/account/api/profile/', {  
  headers: {  
    'Authorization': `Bearer ${data.access}`  
  }  
});
```

